UNIVERSIDAD TECNOLOGICA NACIONAL

FACULTAD REGIONAL SAN FRANCISCO

Tecnicatura Universitaria en Programación

Programación 2

Plan 2024

Eje temático Nº 1

Índice

Unidad Nº 1: Windows Forms

Entorno de desarrollo....4
- Exploración del IDE Visual Studio....4
- Principales accesos directos (Hotkeys)....5

Conceptos básicos....6
- ¿Qué es Windows Forms?....6
  - Características de Windows Forms con C#:....6
- Creación de un proyecto en Windows Forms....7
  - Ejemplo Básico en C#:....7
- Formulario Principal....10

Controles básicos de Windows Forms....11
- Button....11
- Label....11
- TextBox....12
- ComboBox....13
- ListBox....13
- RadioButton....14
- CheckBox....15
- DateTimePicker....15
- ProgressBar....16
- DataGridView....17

Unidad Nº 2:Principios de la Programación Orientada a Objetos (POO)

¿Qué es POO?....18

Clases....18

Propiedades....19

Get y Set (Getter y Setter)....20
- Uso de los getters y setters....22
- Getters y Setters automáticos....22

Constructores....23

Destructores....24

Métodos....25

Abstracción....26

Encapsulamiento....28

Herencia....29

Polimorfismo....30

Unidad Nº 3: Colecciones

¿Qué es una colección?....32
- Características Básicas....32
- Forma de una Clase Colección....32

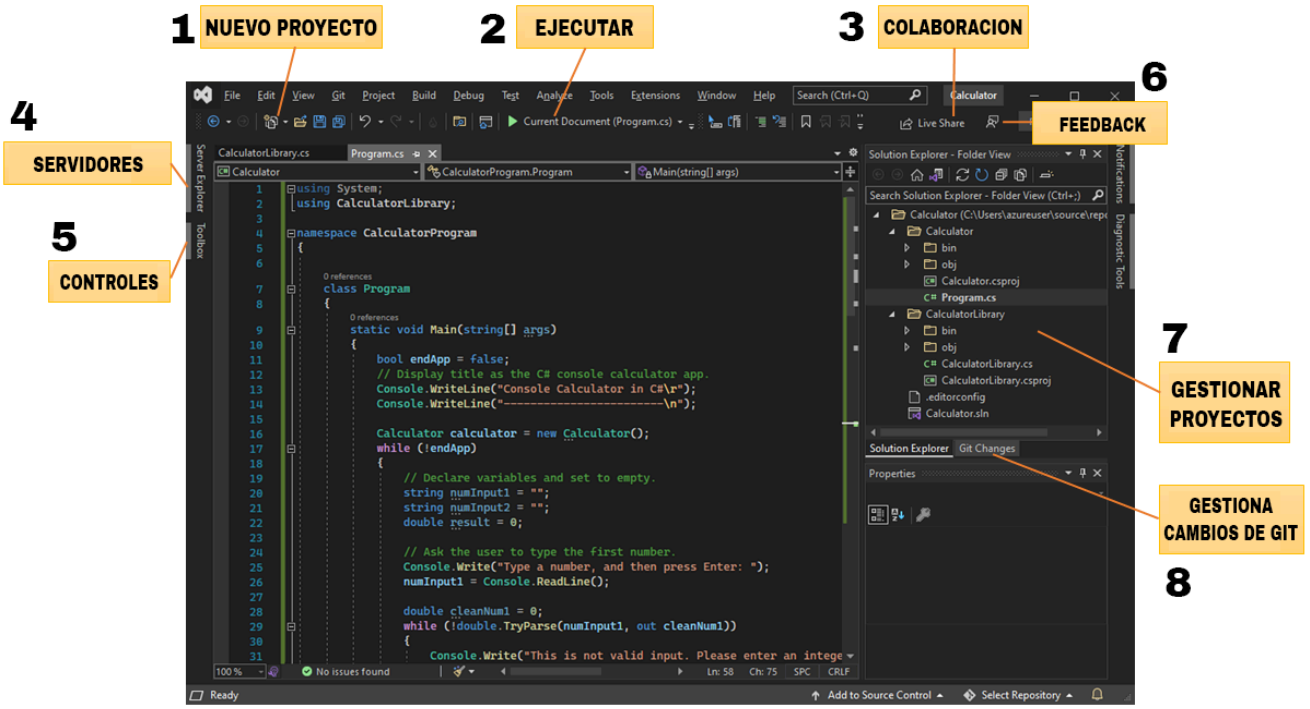
Tipos de colecciones....33

ICollection....34
Métodos y Propiedades Clave:....34
IList....34
Métodos y Propiedades Clave:....34
ArrayList....35
Características de ArrayList:....35
Métodos y Propiedades Clave:....35
Consideraciones al Usar ArrayList:....35
Ejemplos de ArrayList en C#:....36
IDictionary....37
Características de IDictionary:....37
Métodos y Propiedades Clave:....37
HashTable....38
Funcionamiento....38
Ventajas....39
Desventajas....39
Aplicaciones....39
Ejemplo de Hashtable en C#....40
Métodos....41
Agregar Elementos (Add)....41
Determinar la Existencia de Claves y Valores....41
Eliminar Elementos....42

Unidad Nº 1: Windows Forms

Entorno de desarrollo

Exploración del IDE Visual Studio



1. **Nuevo Proyecto**: Permite crear un nuevo conjunto de archivos y configuraciones para iniciar un proyecto desde cero o utilizando plantillas predefinidas.
2. **Ejecutar**: Compila e inicia la ejecución del programa o la aplicación que estás desarrollando para probar su funcionalidad.
3. **Colaboración**: Facilita trabajar en equipo permitiendo la colaboración en tiempo real mediante características como Live Share, que permite a múltiples desarrolladores editar y depurar código simultáneamente desde diferentes ubicaciones.
4. **Explorador de Servidores**: permite explorar y administrar diferentes tipos de servidores y recursos de datos. Administra conexiones a bases de datos, servicios web, servidores de aplicaciones y otros recursos necesarios para el desarrollo de aplicaciones. Permite realizar acciones como explorar estructuras de base de datos, ejecutar consultas, ver y modificar datos, y administrar conexiones y configuraciones de servidor.
5. **Controles (ToolBox)**: Componentes visuales (como botones, cuadros de texto, listas) que se pueden arrastrar y soltar en formularios para diseñar interfaces de usuario en aplicaciones de escritorio o móviles.
6. **Feedback**: Mecanismo para enviar comentarios y sugerencias directamente a Microsoft sobre Visual Studio, sus características y funcionalidades.

7. **Gestión de proyectos y soluciones**: Permite organizar y gestionar múltiples archivos y carpetas relacionados con un proyecto en una solución única, que puede contener varios proyectos interrelacionados.
8. **Gestión de Cambios de Git**: Herramientas integradas para controlar versiones y gestionar cambios en el código fuente utilizando el sistema de control de versiones Git, permitiendo la colaboración y el seguimiento de cambios a lo largo del tiempo.

Principales accesos directos (Hotkeys)

General:

- **Ctrl + S**: Guardar archivo.
- **Ctrl + Shift + S**: Guardar todos los archivos.
- **Ctrl + C**: Copiar.
- **Ctrl + X**: Cortar.
- **Ctrl + V**: Pegar.
- **Ctrl + Z**: Deshacer.
- **Ctrl + Y**: Rehacer.
- **F7**: Ver código fuente
- **SHIFT+ F7:** ver → Diseñador.

Navegación:

- **Ctrl + Tab**: Cambiar entre ventanas de documentos abiertos.
- **Ctrl + -**: Ir hacia atrás (navegación de historia).
- **Ctrl + Shift + -**: Ir hacia adelante (navegación de historia).

Edición y selección:

- **Ctrl + D**: Duplicar línea actual.
- **Ctrl + Space**: Mostrar el Intellisense (sugerencias de código).
- **Ctrl + K, Ctrl + C**: Comentar selección.
- **Ctrl + K, Ctrl + U**: Descomentar selección.

Depuración:

- **F5**: Iniciar depuración.
- **F9**: Establecer/quitar punto de interrupción.
- **F10**: Paso siguiente (sin entrar en funciones).
- **F11**: Paso siguiente (entrar en funciones).

Ventanas y herramientas:

- **Ctrl + Alt + L**: Mostrar/Salir de la ventana de Solution Explorer.
- **Ctrl + Alt + X**: Mostrar/Salir de la ventana de Toolbox (caja de herramientas).

- **Ctrl + Alt + A**: Mostrar/Salir de la ventana de Team Explorer.
- **Ctrl + Alt + S**: Mostrar de la ventana de Explorador de Servidores.

Conceptos básicos

¿Qué es Windows Forms?

Windows Forms (WinForms) es una plataforma de desarrollo de interfaces gráficas de usuario (GUI) para aplicaciones de escritorio en el sistema operativo Windows. Es parte del .NET Framework y se usa principalmente con el lenguaje de programación C#. Windows Forms proporciona un conjunto de controles, componentes y herramientas para crear aplicaciones de Windows con interfaces gráficas ricas y funcionales.

Características de Windows Forms con C#:

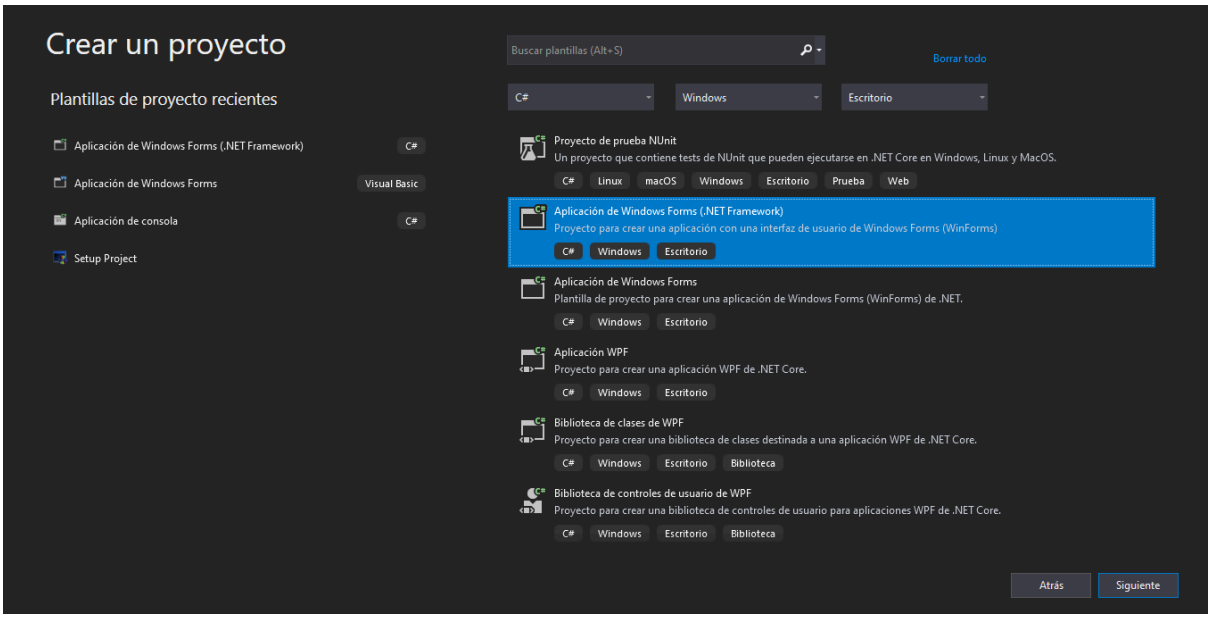
1. **Diseñador Visual:**
    - Permite diseñar la interfaz de usuario de manera visual utilizando el diseñador de formularios de Visual Studio. Puedes arrastrar y soltar controles (como botones, cuadros de texto, etiquetas, etc.) sobre el formulario.
2. **Controles:**
    - Ofrece una amplia gama de controles predefinidos que se pueden personalizar y extender. Estos incluyen botones, cuadros de texto, cuadros de lista, menús, barras de herramientas, y más.
3. **Eventos:**
    - Windows Forms utiliza un modelo basado en eventos. Puedes manejar eventos como clics de botón, movimientos del mouse, teclas presionadas, y otros eventos de la interfaz de usuario escribiendo métodos en C# que respondan a estos eventos.
4. **Interoperabilidad:**
    - Puedes integrar y utilizar componentes COM y DLLs nativas de Windows, lo que permite acceder a una amplia variedad de funcionalidades del sistema operativo.
5. **Acceso a Datos:**
    - Integra fácilmente con ADO.NET y otras tecnologías de acceso a datos, permitiendo crear aplicaciones que interactúan con bases de datos.

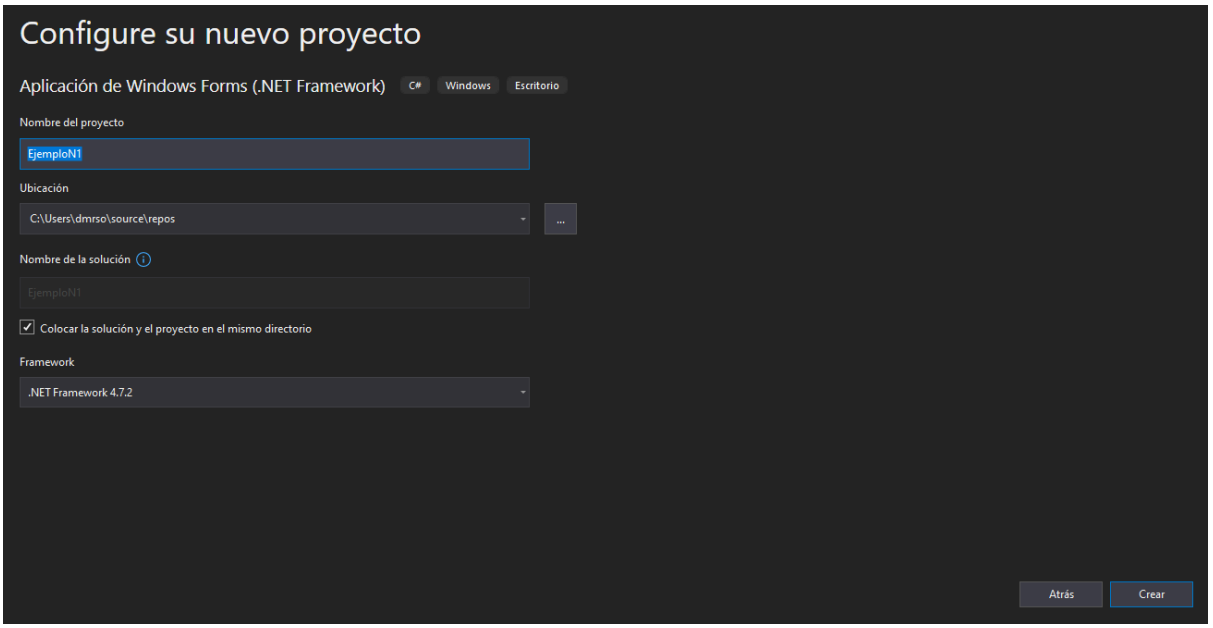
Creación de un proyecto en Windows Forms

Ejemplo Básico en C#:

Crear el Proyecto:

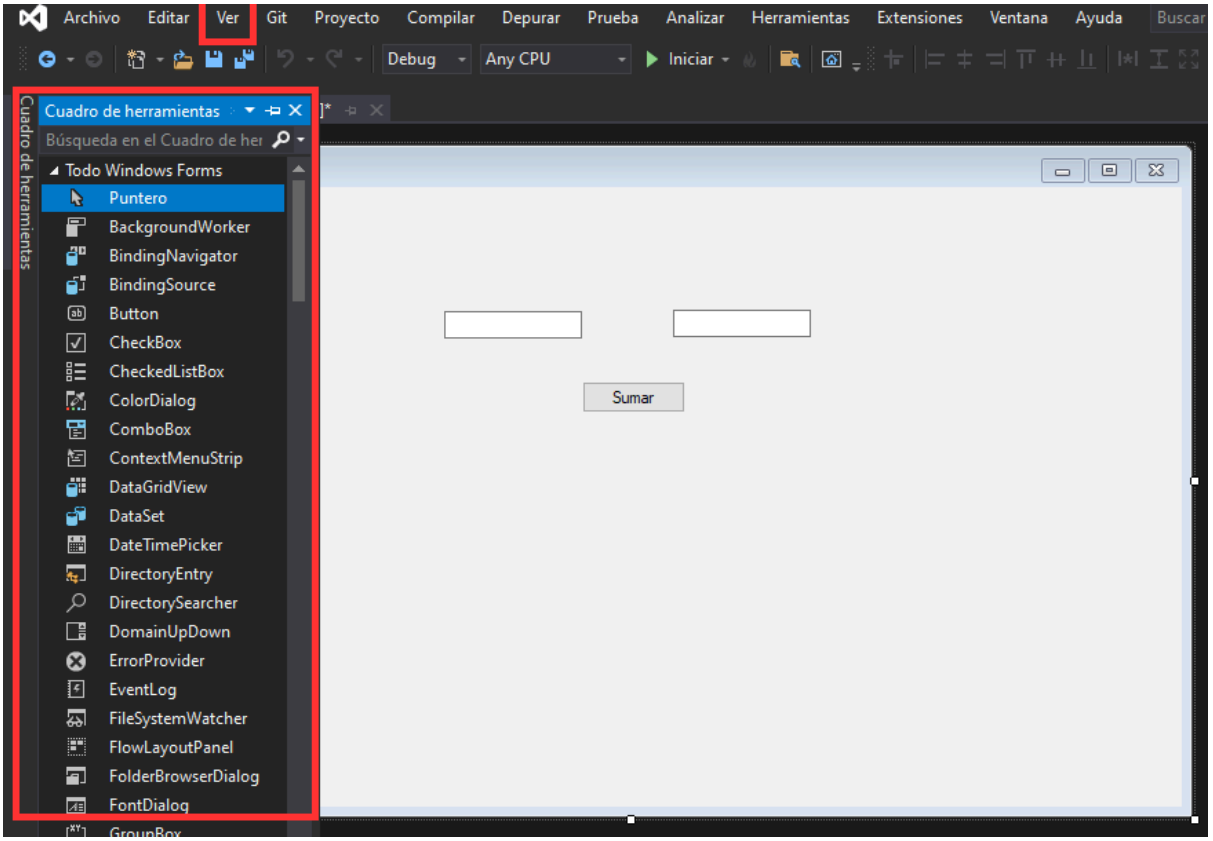
- Abrir Visual Studio.
- Selecciona `Crear un nuevo proyecto`.
- Elegir `Windows Forms App (.NET Framework)` y damos un nombre al proyecto.





Diseñar la Interfaz:

- En el diseñador de formularios (ToolBox o Controles), arrastra dos `TextBox` y un `Button` al formulario. Cambia el texto del botón a `Sumar`.



Agregar el Código:

- Haz doble clic en el botón `Sumar` para generar el método `Click` y agrega el siguiente código:

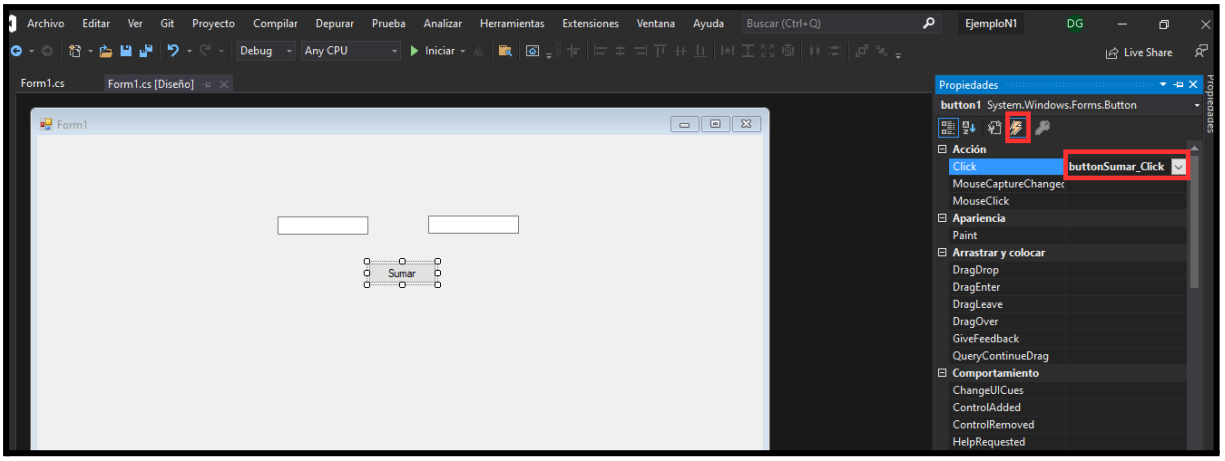
```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace EjemploN1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        private void buttonSumar_Click(object sender, EventArgs e)
        {
            // Obtener los valores de los cuadros de texto
            string text1 = textBox1.Text;
            string text2 = textBox2.Text;
            // Convertir los valores a números
            if ( double.TryParse(text1, out double number1) &&
                 double.TryParse(text2, out double number2)
               )
            {
                double result = number1 + number2;
                // Mostrar el resultado en un MessageBox
                MessageBox.Show($"El resultado de la suma es: {result}", "Resultado");
            }else{
                // Mostrar un mensaje de error si la conversión falla
                MessageBox.Show("Por favor, ingrese números válidos.", "Error");
            }
        }
    }
}
```

`Form1`: Es la clase principal del formulario, generada automáticamente por Visual Studio.

`InitializeComponent()`: Método que inicializa los componentes del formulario. Este método es crucial y es generado automáticamente por Visual Studio cuando diseñas el formulario. Su propósito es configurar las propiedades y los eventos de los controles del formulario y agregar estos controles al formulario mismo.

`buttonSumar_Click`: Método que se ejecuta cuando se hace clic en el botón `Sumar`. Convierte los valores de los `TextBox` a números, realiza la suma y muestra el resultado en un `MessageBox`.

Para modificar el nombre del método, click derecho sobre el button, en propiedades en el evento click le modificamos el nombre.



Formulario Principal

El **formulario principal** de una aplicación Windows Forms es el formulario que se carga y muestra inicialmente cuando se ejecuta la aplicación. Este formulario suele ser la ventana principal de la aplicación y puede contener el menú principal, barras de herramientas, y otros controles de interfaz de usuario que permiten al usuario interactuar con la aplicación.

Propósito del Formulario Principal:

1. **Punto de Entrada de la Aplicación:**
   - Actúa como el punto de entrada visual de la aplicación, donde se muestra la interfaz principal al usuario.
2. **Contenedor de Controles:**
   - Puede contener otros controles y formularios secundarios, como cuadros de diálogo, subformularios, paneles de navegación, etc.
3. **Gestión de Eventos:**
   - Maneja eventos importantes de la aplicación, como la carga inicial, el cierre de la aplicación y otras interacciones clave del usuario.
4. **Coordinación de Funcionalidades:**
   - Coordina la interacción entre diferentes partes de la aplicación, como la gestión de datos, la navegación entre formularios y la comunicación con servicios externos.

Controles básicos de Windows Forms

Button

Es un control gráfico que representa un botón en la interfaz de usuario. Al hacer clic en él, se puede desencadenar una acción o evento específico definido por el desarrollador. Su uso es simple y directo, y se puede manejar fácilmente a través de eventos como `Click`.

Eventos Comunes:

- **`Click`**: Se produce cuando el usuario hace clic en el botón.
- **`MouseEnter`**: Se produce cuando el puntero del ratón entra en el área del botón.
- **`MouseLeave`**: Se produce cuando el puntero del ratón sale del área del botón.
- **`MouseDown`**: Se produce cuando se presiona un botón del ratón mientras el puntero está sobre el botón.
- **`MouseUp`**: Se produce cuando se suelta un botón del ratón mientras el puntero está sobre el botón.

Propiedades Comunes

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Text`**: Texto asociado al control. El texto que visualiza el usuario en el control.
- **`TextAlign`**: Alineación del texto que se mostrará en el control.
- **`Visible`**: Determina si el control está visible u oculto.

Label

El control `Label` en Windows Forms se utiliza para mostrar texto estático en la interfaz de usuario. No permite la edición por parte del usuario y generalmente se utiliza para proporcionar información o descripciones de otros controles en el formulario, como `TextBox`, `Button`, `ComboBox`, etc.
El tamaño y la posición del `Label` en el formulario se pueden ajustar a través de las propiedades `Size` y `Location`. Puedes personalizar la apariencia del texto utilizando propiedades como `Font`, `ForeColor`, `BackColor`, `TextAlign`, etc.

Propiedades Comunes

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.

- **`Autosize`**: Habilita cambio de tamaño automático según el tamaño de la fuente.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Text`**: Texto asociado al control. El texto que visualiza el usuario en el control.
- **`TextAlign`**: Alineación del texto que se mostrará en el control.
- **`Visible`**: Determina si el control está visible u oculto.

TextBox

El control TextBox en Windows Forms es utilizado para recibir entrada de texto por parte del usuario. Es uno de los controles más fundamentales y versátiles para capturar información en una aplicación de escritorio. Es útil para formularios, búsquedas, y cualquier situación donde se requiera capturar datos del usuario. Se pueden personalizar su apariencia y comportamiento mediante propiedades como Text, ForeColor, BackColor, Font, Multiline (permite al `TextBox` manejar múltiples líneas de texto), PasswordChar (para enmascarar la entrada del usuario, útil para contraseñas y datos sensibles), etc.

Varios eventos como `TextChanged`, `KeyPress`, `KeyDown`, `KeyUp` permiten manejar la entrada del usuario y realizar validaciones en tiempo real.

`TextChanged`: Se produce cuando cambia el contenido del texto.
`KeyPress`: Se produce cuando se presiona una tecla mientras el `TextBox` tiene el foco.
`KeyDown`: Se produce cuando se presiona una tecla.
`KeyUp`: Se produce cuando se libera una tecla.

Propiedades Comunes

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Text`**: Texto asociado al control. El texto que se visualiza en el control.
- **`TextAlign`**: Alineación del texto que se mostrará en el control.
- **`Visible`**: Determina si el control está visible u oculto.

ComboBox

El control `ComboBox` en Windows Forms es un control de selección que combina las características de un `TextBox` y una lista desplegable. Permite al usuario seleccionar un elemento de una lista predefinida o ingresar un valor directamente.

Eventos Comunes:

- **`SelectedIndexChanged`**: Se produce cuando el índice seleccionado cambia.
- **`TextChanged`**: Se produce cuando cambia el texto del `ComboBox`.
- **`DropDown`**: Se produce cuando se abre la lista desplegable.
- **`DropDownClosed`**: Se produce cuando se cierra la lista desplegable.

Propiedades Comunes

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Text`**: Texto asociado al control. El texto que visualiza el usuario en el control.
- **`TextAlign`**: Alineación del texto que se mostrará en el control.
- **`Visible`**: Determina si el control está visible u oculto.
- **`DisplayMember`**: Indica la propiedad que se va a mostrar.
- **`ValueMember`**: indica la propiedad que se va a utilizar como valor real.
- **`DataSource`**: Indica la lista que este control utiliza para obtener sus elementos.

ListBox

El control `ListBox` en Windows Forms se utiliza para mostrar una lista de elementos de los cuales el usuario puede seleccionar uno o varios. Es útil para mostrar listas largas de opciones y permite una fácil manipulación de los elementos.
Permite la selección de uno o varios elementos dependiendo de la configuración de la propiedad `SelectionMode`. Muestra una lista de elementos que puede ser desplazada si el número de elementos excede el tamaño del `ListBox`.

Eventos Comunes:

- **`SelectedIndexChanged`**: Se produce cuando el índice seleccionado cambia.
- **`SelectedValueChanged`**: Se produce cuando el valor seleccionado cambia.
- **`Click`**: Se produce cuando se hace clic en el `ListBox`.

- **`DoubleClick`**: Se produce cuando se hace doble clic en un elemento del `ListBox`.

Propiedades Comunes

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Text`**: Texto asociado al control. El texto que visualiza el usuario en el control.
- **`TextAlign`**: Alineación del texto que se mostrará en el control.
- **`Visible`**: Determina si el control está visible u oculto.
- **`DisplayMember`**: Indica la propiedad que se va a mostrar.
- **`ValueMember`**: indica la propiedad que se va a utilizar como valor real.
- **`DataSource`**: Indica la lista que este control utiliza para obtener sus elementos.
- **`Items`**: elementos en el cuadro de lista.
- **`SelectionMode`**: Indica si permite selección simple o múltiple.

RadioButton

El control `RadioButton` en Windows Forms se utiliza para permitir que el usuario seleccione una opción entre un conjunto de opciones mutuamente excluyentes. Generalmente, se agrupan varios `RadioButton` juntos para que solo uno de ellos pueda estar seleccionado a la vez dentro de un grupo.

Eventos Comunes:

`CheckedChanged`: Se produce cuando el valor de la propiedad `Checked` cambia.

Propiedades Comunes

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Text`**: Texto asociado al control. El texto que visualiza el usuario en el control.
- **`TextAlign`**: Alineación del texto que se mostrará en el control.
- **`Visible`**: Determina si el control está visible u oculto.
- **`Checked`**: Indica si el objeto está activado o no.

CheckBox

El control `CheckBox` en Windows Forms se utiliza para permitir que el usuario seleccione o deseleccione una opción. Puede ser utilizado para representar estados booleanos (verdadero/falso) y puede ser independiente o parte de un grupo de `CheckBox` donde múltiples opciones pueden estar seleccionadas al mismo tiempo.

Eventos Comunes:

- **`CheckedChanged:`** Se produce cuando el valor de la propiedad Checked cambia.
- **`CheckStateChanged:`** Se produce cuando el valor de la propiedad `CheckState` cambia.

Propiedades Comunes:

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Text`**: Texto asociado al control. El texto que visualiza el usuario en el control.
- **`TextAlign`**: Alineación del texto que se mostrará en el control.
- **`Visible`**: Determina si el control está visible u oculto.
- **`Checked`**: Indica si el objeto está activado o no.

DateTimePicker

El control `DateTimePicker` en Windows Forms se utiliza para permitir que el usuario seleccione una fecha, una hora, o ambas. Proporciona una interfaz fácil de usar para la selección de fechas y horas, con un calendario desplegable y la capacidad de ingresar manualmente valores.

Eventos Comunes:

`ValueChanged`: Se produce cuando el valor de la propiedad `Value` cambia.

Propiedades Comunes

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.

- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`TabIndex`**: Determina el índice del orden de tabulación que ocupa el control.
- **`Visible`**: Determina si el control está visible u oculto.
- **`Value`**: Valor de fecha y hora actual del control.

ProgressBar

El control `ProgressBar` en Windows Forms se utiliza para mostrar visualmente el progreso de una operación en curso. Es comúnmente utilizado para indicar al usuario el avance de tareas como la carga de datos, la descarga de archivos, o el procesamiento de información.

Eventos Comunes:

- **`Click:`** Se produce cuando el usuario hace clic en la barra de progreso.
- **`ValueChanged:`** Se produce cuando cambia el valor de progreso (`Value`).

Propiedades Comunes:

- **`Name`**: Indica el nombre utilizado en el código para identificar el objeto.
- **`Enabled`**: Indica si el control está habilitado o no para que el usuario interactúe.
- **`Location`**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **`Size`**: Tamaño del control en píxeles.
- **`Visible`**: Determina si el control está visible u oculto.
- **`Maximum`**: Límite superior del intervalo con el que trabaja el control.
- **`Minimum`**: Límite inferior del intervalo con el que trabaja el control.

DataGridView

El control `DataGridView` en Windows Forms se utiliza para mostrar y editar datos en una cuadrícula de filas y columnas. Ofrece una amplia gama de funcionalidades para la manipulación de datos y una interfaz altamente configurable.

Eventos Comunes:

- **CellClick**: Se produce cuando el usuario hace clic en una celda.
- **CellValueChanged**: Se produce cuando el valor de una celda cambia.
- **RowEnter**: Se produce cuando el usuario entra en una fila.
- **RowLeave**: Se produce cuando el usuario sale de una fila.
- **SelectionChanged**: Se produce cuando la selección de celdas cambia.
- **CellFormatting**: Se produce cuando se formatea una celda.
- **CellValidating**: Se produce cuando una celda está a punto de perder el foco y se valida el contenido.
- **DataError**: Se produce cuando ocurre un error en el procesamiento de datos.

Propiedades Comunes:

- **Name**: Indica el nombre utilizado en el código para identificar el objeto.
- **Enabled**: Indica si el control está habilitado o no para que el usuario interactúe.
- **Location**: Coordenadas de la esquina superior izquierda del control respecto a la esquina superior izquierda de su contenedor.
- **Size**: Tamaño del control en píxeles.
- **TabIndex**: Determina el índice del orden de tabulación que ocupa el control.
- **Visible**: Determina si el control está visible u oculto.
- **DataSource**: Fuente de datos que proporciona los datos a mostrar en el `DataGridView`..
- **AllowUserToAddRows**: Indica si se permite al usuario añadir nuevas filas (por defecto es `true`).
- **AllowUserToDeleteRows**: Indica si se permite al usuario eliminar filas existentes (por defecto es `true`).
- **AllowUserToOrderColumns**: Indica si se permite al usuario ordenar las columnas arrastrando sus cabeceras (por defecto es `true`).
- **AllowUserToResizeColumns**: Indica si se permite al usuario cambiar el tamaño de las columnas (por defecto es `true`).
- **AllowUserToResizeRows**: Indica si se permite al usuario cambiar el tamaño de las filas (por defecto es `true`).
- **ColumnHeadersHeightSizeMode**: Determina cómo se ajusta la altura de los encabezados de columna (puede ser `AutoSize`, `DisableResizing`, o `EnableResizing`).
- **RowHeadersWidthSizeMode**: Determina cómo se ajusta el ancho de los encabezados de fila (puede ser `AutoSize`, `DisableResizing`, o `EnableResizing`).

Unidad Nº 2:
Principios de la Programación Orientada a Objetos (POO)

¿Qué es POO?

La POO es un método de implementación en el que los programas se organizan como colecciones cooperativas de objetos, cada uno de los cuales representan una instancia de alguna clase, y cuyas clases son todas miembros de una jerarquía de clases unidas mediante relaciones de herencia .

Otras definiciones:
La POO es un paradigma de programación en el que se utilizan objetos que se comunican a través de mensajes enviados entre éstos. La POO es una forma especial de programar, más cercana a cómo expresamos las cosas en la vida real.

Clases

Son plantillas que definen la estructura y el comportamiento de los objetos en programación orientada a objetos. Representan entidades abstractas que encapsulan datos (campos) y operaciones (métodos). A todos los objetos que vayamos a crear bajo esta “plantilla” (clase), los vamos a llamar **instancias** de dicha clase.

```
// Definición de una clase 'Persona'
public class Persona
{
    // Campos
    public string Nombre;
    public int Edad;

    // Método
    public void Saludar()
    {
        Console.WriteLine("Hola, soy " + Nombre + " y tengo " + Edad + " años.");
    }
}

// Uso de la clase Persona
class Program
{
    static void Main()
    {
        Persona persona1 = new Persona();
        persona1.Nombre = "Juan";
        persona1.Edad = 30;
        persona1.Saludar();  // Salida: Hola, soy Juan y tengo 30 años.
    }
}
```

Propiedades

Como puede observarse en el ejemplo de la clase **Persona**, creamos dos variables public string Nombre y public int Edad. En este entorno dentro de la clase, a estas variables las vamos a llamar propiedades de la misma. Al estar creando una clase que describa a una persona, los datos relevantes a ella misma, son propios de esa persona, como en la vida misma.

Cabe destacar que solo podemos acceder a las propiedades públicas (variables declaradas con **public**) definidas en la clase. Para acceder a las variables privadas, debemos utilizar **get** y **set**.

Si ponemos como ejemplo crear una clase Mesa, puede tener como propiedades public int cantidadDePatas, public string material, etc.

Una vez creado un objeto a partir de la clase, adoptará todos los métodos y propiedades de su clase creadora. Entonces para acceder a ellos, utilizaremos el punto como conector:

```
Persona persona1 = new Persona();
persona1.Nombre = "Juan"; // Acceso a la propiedad Nombre de la clase Persona
persona1.Edad = 30; // Acceso a la propiedad Edad de la clase Persona
persona1.Saludar(); // Acceso al método Saludar de la clase Persona
```

Get y Set (Getter y Setter)

En programación orientada a objetos, los getter y setter son métodos utilizados para acceder y modificar el valor de variables privadas (campos) dentro de una clase. Estos métodos proporcionan un mecanismo controlado para leer y escribir valores, aplicando el principio de encapsulamiento al restringir el acceso directo a los datos internos de un objeto.

Supongamos que tenemos una clase Persona que tiene campos privados para Nombre y Edad, y queremos acceder a estos campos mediante getter y setter:

```
public class Persona
{
    private string nombre;
    private int edad;

    // Getter para obtener el valor del campo 'nombre'
    public string GetNombre()
    {
        return nombre;
    }

    // Setter para asignar un valor al campo 'nombre'
    public void SetNombre(string nuevoNombre)
    {
        nombre = nuevoNombre;
    }

    // Getter para obtener el valor del campo 'edad'
    public int GetEdad()
    {
        return edad;
    }

    // Setter para asignar un valor al campo 'edad'
    public void SetEdad(int nuevaEdad)
    {
        if (nuevaEdad > 0)
        {
            edad = nuevaEdad;
        }
        else
        {
            Console.WriteLine("Edad no válida");
        }
    }
}
```

- Se define la clase Persona con campos privados (nombre y edad).
- Se proporcionan métodos públicos (GetNombre, SetNombre, GetEdad, SetEdad) para acceder y modificar estos campos.
- Los métodos GetNombre y GetEdad son getters que devuelven el valor actual de los campos nombre y edad, respectivamente.
- Los métodos SetNombre y SetEdad son setters que permiten asignar nuevos valores a los campos nombre y edad, respectivamente. El setter para la edad realiza una validación para asegurarse de que el valor asignado sea mayor que cero.

Uso de los getters y setters

Siguiendo el ejemplo anterior, consumimos las propiedades de la clase **Persona** a través de los Setters y Getters creados.

```
class Program
{
    static void Main()
    {
        Persona persona1 = new Persona();

        // Usando el setter para asignar valores
        persona1.SetNombre("Juan");
        persona1.SetEdad(30);

        // Usando el getter para obtener valores
        Console.WriteLine($"Nombre: {persona1.GetNombre()}");
        Console.WriteLine($"Edad: {persona1.GetEdad()}");
    }
}
```

De esta manera conservamos la privacidad de la estructura de la clase y tenemos control de la escritura y lectura de nuestras variables privadas. Algunas de las aplicaciones que podemos darle, puede ser filtrar o limpiar los valores que recibimos desde afuera de la clase o así, como convertir a otro formato el valor que vamos a devolver.

Getters y Setters automáticos

Otra forma de definirlos, es a través de la auto implementación de su propiedad:

```
public string Nombre { get; set; }
```

Constructores

Son métodos especiales de una clase que se llaman automáticamente al crear una instancia (objeto) de esa clase. Se utilizan para inicializar los campos y establecer el estado inicial del objeto.

```
public class Persona
{
    public string Nombre;
    public int Edad;

    // Constructor
    public Persona(string nombre, int edad)
    {
        Nombre = nombre;
        Edad = edad;
    }

    // Método
    public void Saludar()
    {
        Console.WriteLine("Hola, soy " + Nombre + " y tengo " + Edad + " años.");
    }
}

class Program
{
    static void Main()
    {
        Persona persona1 = new Persona("Ana", 25);
        persona1.Saludar();  // Salida: Hola, soy Ana y tengo 25 años.
    }
}
```

Destructores

En C#, los destructores (finalizadores) se utilizan para liberar recursos no administrados o realizar otras tareas de limpieza antes de que el objeto sea destruido y liberado de la memoria. No se suelen utilizar explícitamente con la misma frecuencia que los constructores.

```
public class MiClase
{
    // Destructor
    ~MiClase()
    {
        // Código para liberar recursos no administrados
        Console.WriteLine("Objeto destruido.");
    }
}

class Program
{
    static void Main()
    {
        MiClase instancia = new MiClase();
        instancia = null;  // Provoca que el destructor sea invocado
        GC.Collect();  // Forzar la recolección de basura
    }
}
```

Métodos

Los métodos son funciones que están definidas dentro de una clase y se utilizan para realizar acciones específicas sobre los objetos de esa clase. Pueden tomar parámetros y devolver valores, o sea que pueden ser Funciones o Procedimientos. Estos, al estar dentro de una Clase, los llamaremos Métodos para entender que pertenecen a alguna clase y no están sueltos en el código.

```
public class Calculadora
{
    public int Sumar(int a, int b)
    {
        return a + b;
    }
}

class Program
{
    static void Main()
    {
        Calculadora calc = new Calculadora();
        int resultado = calc.Sumar(5, 3);  // resultado = 8
        Console.WriteLine("La suma es: " + resultado);
    }
}
```

En este ejemplo, vemos cómo realizamos una instancia de la clase **Calculadora** en el objeto **calc**. Luego llamamos a su función **Sumar** a la cual le enviamos 2 parámetros para que nos devuelva el valor y asignarlo a la variable resultado.

Abstracción

La abstracción es el proceso de identificar las características esenciales de un objeto y ocultar los detalles innecesarios para enfocarse en lo que es importante desde una perspectiva del problema.

Esta acción es también conocida como modelizar algo de la vida real, en el formato de una clase. La abstracción permite representar conceptos del mundo real en el contexto de programación mediante clases, que encapsulan características y comportamientos relevantes.

Imaginemos que tenemos que aplicar abstracción sobre las aves. El objeto sería el pájaro y sus características (propiedades) serían: pico, alas, plumas, patas. Por otro lado las funcionalidades que puede tener son: Volar, Parar, Saltar, etc.

```
// Clase que representa una abstracción de un Ave
public class Ave
{
    // Propiedades con encapsulación
    public string Pico { get; set; }
    public string Alas { get; set; }
    public string Plumas { get; set; }
    public string Patas { get; set; }

    // Método para volar
    public void Volar()
    {
        // Código para volar...
        Console.WriteLine("El ave está volando.");
    }

    // Método para parar
    public void Parar()
    {
        // Código para detener el vuelo...
        Console.WriteLine("El ave ha parado de volar.");
    }

    // Método para saltar
    public void Saltar()
    {
        // Código para saltar...
        Console.WriteLine("El ave está saltando.");
    }
}
```

Para poder utilizar esta clase, crearemos una instancia (objeto) de esa clase.

```
// Clase principal para probar la abstracción
public class Programa
{
    public static void Main(string[] args)
    {
        // Crear una instancia de Ave
        Ave pajaro = new Ave();
        pajaro.Pico = "Pico afilado";
        pajaro.Alas = "Alas largas";
        pajaro.Plumas = "Plumas coloridas";
        pajaro.Patas = "Patas fuertes";

        // Usar los métodos de la clase Ave
        pajaro.Volar();  // Salida: El ave está volando.
        pajaro.Parar();  // Salida: El ave ha parado de volar.
        pajaro.Saltar(); // Salida: El ave está saltando.
    }
}
```

Encapsulamiento

Es el principio de ocultar los detalles internos de cómo funciona un objeto y restringir el acceso directo a ellos, proporcionando acceso controlado a través de métodos y propiedades.

```
public class CuentaBancaria
{
    private decimal saldo;  // Campo privado

    public void Depositar(decimal cantidad)
    {
        saldo += cantidad;
    }

    public decimal ObtenerSaldo()
    {
        return saldo;
    }
}

class Program
{
    static void Main()
    {
        CuentaBancaria cuenta = new CuentaBancaria();
        cuenta.Depositar(1000);
        decimal saldoActual = cuenta.ObtenerSaldo();
        Console.WriteLine("Saldo actual: " + saldoActual);  // Salida: Saldo actual: 1000
    }
}
```

De esta manera al realizar un depósito, no sabemos qué procedimientos realiza la clase y también ocultamos el nombre de la variable encargada de guardar el saldo, donde para obtener su valor, debemos recurrir al método **ObtenerSaldo().**

Herencia

Permite que una clase (subclase o clase derivada) herede atributos y métodos de otra clase (superclase o clase base). Esto promueve la reutilización del código y permite la extensión de la funcionalidad existente.

```
public class Animal
{
    public void Dormir()
    {
        Console.WriteLine("Zzz...");
    }
}

public class Perro : Animal
{
    public void EmitirSonido()
    {
        Console.WriteLine("¡Guau guau!");
    }
}

public class Gato : Animal
{
    public void EmitirSonido()
    {
        Console.WriteLine("¡Miau miau!");
    }
}

class Program
{
    static void Main()
    {
        Perro miPerro = new Perro();
        miPerro.Dormir();  // Salida: Zzz...
        miPerro.EmitirSonido();  // Salida: ¡Guau guau!

        Gato miGato = new Gato();
        miGato.Dormir();  // Salida: Zzz...
        miGato.EmitirSonido();  // Salida: ¡Guau guau!
    }
}
```

En este ejemplo, ambas clases **Perro** y **Gato**, heredan de la clase **Animal**. Allí reciben todos los métodos y propiedades de **Animal**. En este caso, solo el método **Dormir**. Esto tiene sentido ya que ambos animales pueden dormir, pero no emiten sonido de la misma

manera. A diferencia de la Abstracción, los métodos **EmitirSonido** no son obligatorios de definir ya que en la clase padre, no tiene la especificación **abstract**.

Polimorfismo

El polimorfismo permite que un mismo método o propiedad tenga diferentes implementaciones en diferentes clases. Se puede lograr mediante la sobreescritura de métodos (en el caso de polimorfismo de tiempo de ejecución) o mediante la sobrecarga de métodos (en el caso de polimorfismo de tiempo de compilación).

```
public class Figura
{
    public virtual void Dibujar()
    {
        Console.WriteLine("Dibujando una figura genérica");
    }
}

public class Circulo : Figura
{
    public override void Dibujar()
    {
        Console.WriteLine("Dibujando un círculo");
    }
}

public class Cuadrado : Figura
{
    public override void Dibujar()
    {
        Console.WriteLine("Dibujando un cuadrado");
    }
}

class Program
{
    static void Main()
    {
        Figura figura1 = new Circulo();
        Figura figura2 = new Cuadrado();

        figura1.Dibujar();  // Salida: Dibujando un círculo
        figura2.Dibujar();  // Salida: Dibujando un cuadrado
    }
}
```

Cuando un método o propiedad se marca como **virtual** en la clase base, se indica que este método puede ser sobrescrito en las clases derivadas. Esto permite que las clases hijas reemplacen la implementación del método base con su propia implementación mediante **override**. En el lenguaje C#, esto es fundamental para poder implementar las clases según sea necesario.

Unidad Nº 3: Colecciones

¿Qué es una colección?

Una colección es una clase o un objeto que agrupa un conjunto de elementos de tipos similares, ofreciendo métodos y propiedades para gestionar estos elementos de manera eficiente.

En C#, una colección es un objeto que agrupa múltiples elementos de tipos similares, proporcionando métodos y propiedades para su manipulación, acceso y gestión. Las colecciones se implementan a través de clases e interfaces del espacio de nombres System.Collections o System.Collections.Generic.

Características Básicas

- **Capacidad**: Número máximo de elementos que una colección puede contener antes de necesitar expandirse. En colecciones como `List<T>`, la capacidad puede gestionarse dinámicamente.
- **Tamaño**: Número actual de elementos en la colección. En `List<T>`, se accede a través de la propiedad `Count`.
- **Expansión Automática**: Las colecciones como `List<T>` aumentan automáticamente su capacidad cuando se alcanza el límite actual, utilizando métodos como `Add` para añadir nuevos elementos.
- **Índice Inferior**: Es el índice del primer elemento de la colección, típicamente cero. Se accede a los elementos mediante índices, como en `myList[0]`.

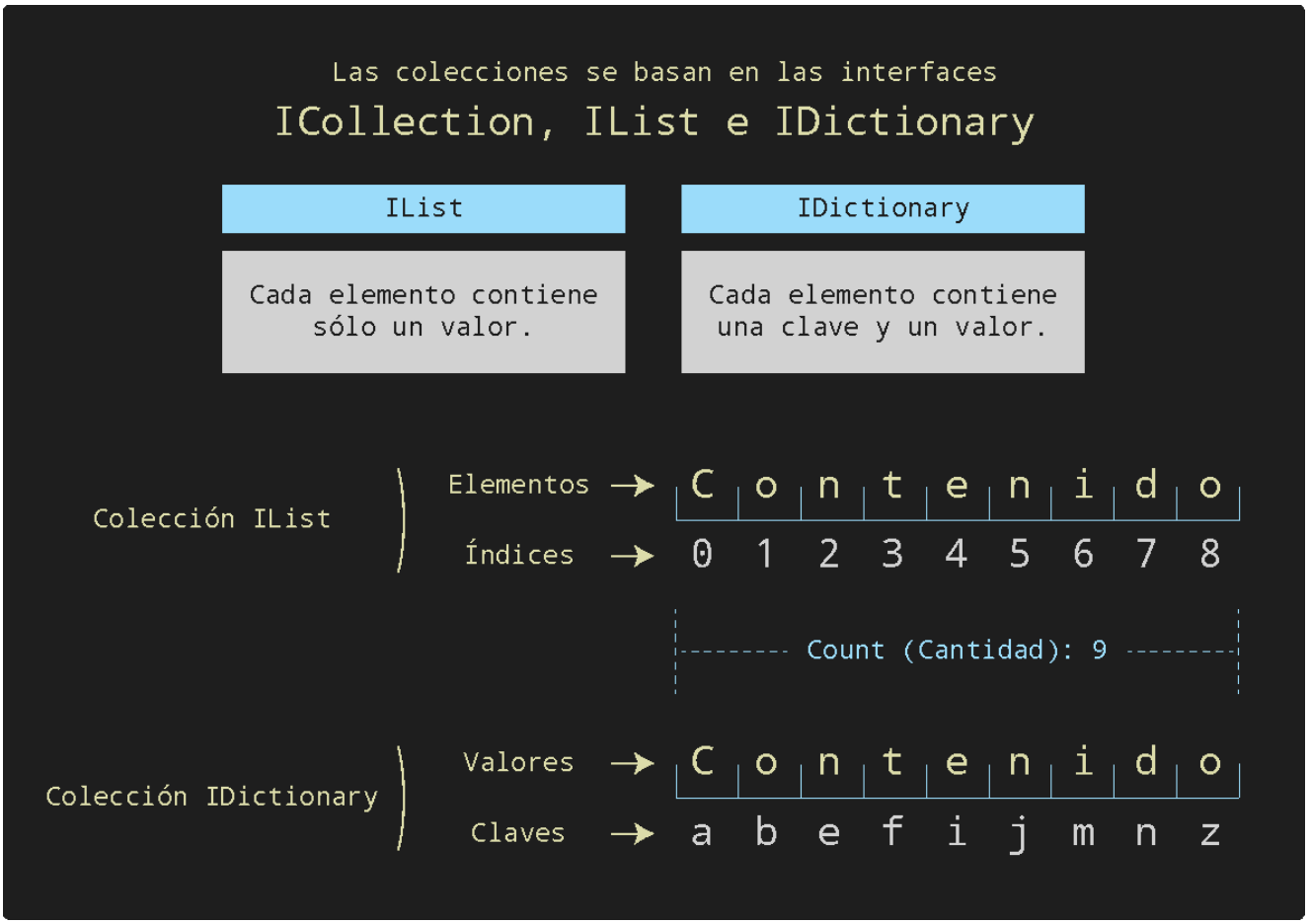
Forma de una Clase Colección

La forma de una clase *colección* se refiere a la manera en que organiza y almacena los objetos. A continuación se presentan las diferentes formas que puede adoptar una colección:

- **Lista**: Proporciona una secuencia de elementos ordenados. Los elementos no están asociados a índices específicos, pero se pueden recorrer en el orden en que se añadieron. Ejemplo: `List<T>` en C#.
- **Matriz (Array)**: Proporciona una estructura de datos de tamaño fijo o dinámico, donde los elementos se almacenan en ubicaciones contiguas y se accede a ellos mediante índices enteros. Ejemplo: `T[]` en C# para matrices de tamaño fijo y `Array` para matrices dinámicas.
- **Mapa (Map)**: Es una colección que asocia claves únicas a valores. Permite acceder a un valor específico a través de su clave correspondiente. Ejemplo: `Dictionary<TKey, TValue>` en C#.

- **Ordenada (Ordered Collection)**: Mantiene los elementos en un orden específico determinado por el orden de inserción o por un criterio de ordenación definido. Ejemplo: `SortedList<TKey, TValue>` y `SortedDictionary<TKey, TValue>` en C#.
- **Indizada (Indexed Collection)**: Los elementos pueden ser recuperados mediante un índice entero, similar a una matriz. Permite acceso rápido y directo a cualquier elemento. Ejemplo: `List<T>` en C#.

Tipos de colecciones



ICollection

Es una interfaz en el espacio de nombres System.Collections.Generic que define una colección genérica de objetos. Es la interfaz base para muchas otras colecciones en .NET, proporcionando funcionalidad básica para manipular y recorrer colecciones de elementos.

Métodos y Propiedades Clave:

- **`int Count`**: Obtiene el número de elementos contenidos en la colección.
- **`void Add(T item)`**: Agrega un elemento a la colección.
- **`void Clear()`**: Elimina todos los elementos de la colección.
- **`bool Contains(T item)`**: Determina si la colección contiene un elemento específico.
- **`void CopyTo(T[] array, int arrayIndex)`**: Copia los elementos de la colección a una matriz, comenzando en un índice de matriz en particular.
- **`bool Remove(T item)`**: Elimina la primera aparición de un objeto específico de la colección.

IList

`IList` es una interfaz en C# que representa una colección de elementos que pueden ser accedidos mediante un índice. Es una subinterfaz de `ICollection` y `IEnumerable`, lo que significa que hereda los métodos de estas interfaces para la manipulación y enumeración de elementos.

Características Principales:

- **Acceso Indexado**: Permite acceder, añadir, eliminar y modificar elementos utilizando índices enteros.
- **Ordenado**: Mantiene los elementos en el orden en que fueron añadidos.
- **Modificable**: Se pueden añadir, eliminar o modificar elementos en cualquier posición.

Métodos y Propiedades Clave:

- **`int Count`**: Obtiene el número de elementos en la lista
- **`void Add(T item)`**: Agrega un elemento a la lista.
- **`bool Remove(T item)`**: Elimina la primera aparición de un objeto específico de la lista.
- **`void Insert(int index, T item)`**: Inserta un elemento en el índice especificado.
- **`int IndexOf(T item)`**: Devuelve el índice de la primera aparición de un objeto específico.

- **`void RemoveAt(int index)`**: Elimina el elemento en el índice especificado.

ArrayList

Es una clase en el espacio de nombres **System**.Collections que representa una colección de objetos cuyo tamaño puede aumentar o disminuir dinámicamente. A diferencia de las colecciones genéricas **(List<T>)**, ArrayList puede contener elementos de cualquier tipo.

Características de `ArrayList`:

- **Capacidad Dinámica**: **`ArrayList`** ajusta automáticamente su capacidad interna a medida que se agregan o eliminan elementos.
- **Acceso por Índice**: Los elementos pueden accederse mediante índices, similar a una matriz.
- **No Genérico**: Puede contener elementos de cualquier tipo, lo que significa que no ofrece seguridad de tipos.

Métodos y Propiedades Clave:

- **`int Count`**: Obtiene el número de elementos que contiene el `ArrayList`.
- **`int Capacity`**: Obtiene o establece el número de elementos que el `ArrayList` puede contener.
- **`void Add(object value)`**: Agrega un objeto al final del `ArrayList`.
- **`void Remove(object value)`**: Elimina la primera aparición de un objeto específico.
- **`void RemoveAt(int index)`**: Elimina el elemento en el índice especificado.
- **`bool Contains(object item)`**: Determina si un elemento específico está en el `ArrayList`.
- **`void Clear()`**: Elimina todos los elementos del `ArrayList`.
- **`int IndexOf(object value)`**: Devuelve el índice de la primera aparición de un objeto específico.

Consideraciones al Usar `ArrayList`:

- **Rendimiento**: `ArrayList` puede ser menos eficiente que las colecciones genéricas debido a la necesidad de convertir tipos de manera explícita.
- **Seguridad de Tipos**: `ArrayList` no proporciona seguridad de tipos en tiempo de compilación, lo que puede llevar a errores de tiempo de ejecución.
- **Compatibilidad**: Aunque `ArrayList` sigue estando disponible, se recomienda utilizar `List<T>` para colecciones de tipo específico por razones de rendimiento y seguridad de tipos.

Ejemplos de ArrayList en C#:

```
using System;
using System.Collections;

public class Program
{
    public static void Main()
    {
        // Crear una instancia de ArrayList
        ArrayList arrayList = new ArrayList();

        // Agregar elementos de diferentes tipos
        arrayList.Add(1);
        arrayList.Add("dos");
        arrayList.Add(3.0);
        arrayList.Add(DateTime.Now);

        // Mostrar los elementos del ArrayList
        foreach (var item in arrayList)
        {
            Console.WriteLine(item);
        }

        // Acceder a un elemento por índice
        Console.WriteLine("Elemento en el índice 1: " + arrayList[1]);

        // Verificar si un elemento está en el ArrayList
        bool contieneDos = arrayList.Contains("dos");
        Console.WriteLine("Contiene 'dos': " + contieneDos);

        // Eliminar un elemento
        arrayList.Remove("dos");

        // Mostrar los elementos después de la eliminación
        Console.WriteLine("Elementos después de eliminar 'dos':");
        foreach (var item in arrayList)
        {
            Console.WriteLine(item);
        }

        // Eliminar un elemento en el índice especificado
        arrayList.RemoveAt(0);

        // Mostrar los elementos después de la eliminación por índice
        Console.WriteLine("Elementos después de eliminar el elemento en el índice 0:");
        foreach (var item in arrayList)
        {
            Console.WriteLine(item);
        }

        // Limpiar el ArrayList
        arrayList.Clear();
        Console.WriteLine("Cantidad de elementos después de Clear: " + arrayList.Count);
    }
}
```

IDictionary

`IDictionary` es una interfaz en el espacio de nombres **`System.Collections.Generic`** que define una colección de pares clave-valor. Es ideal para almacenar datos en los que cada valor se asocia con una clave única. **`IDictionary`** hereda de **`ICollection`** y **`IEnumerable`**.

Características de `IDictionary`:

- **Asociación Clave-Valor**: Cada elemento se almacena como un par clave-valor.
- **Clave Única**: Las claves deben ser únicas dentro del diccionario.
- **Acceso por Clave**: Los valores se pueden recuperar, agregar o eliminar utilizando las claves.

Métodos y Propiedades Clave:

- **`int Count`**: Obtiene el número de pares clave-valor en el diccionario.
- **`bool IsReadOnly`**: Indica si el diccionario es de solo lectura.
- **`TValue this[TKey key] { get; set; }`**: Permite acceder o asignar un valor asociado con una clave específica.
- **`void Add(TKey key, TValue value)`**: Agrega un par clave-valor al diccionario.
- **`bool Remove(TKey key)`**: Elimina el par clave-valor con la clave especificada.
- **`bool ContainsKey(TKey key)`**: Determina si el diccionario contiene una clave específica.
- **`bool TryGetValue(TKey key, out TValue value)`**: Obtiene el valor asociado con la clave especificada, si existe.

HashTable

Un hashtable es una estructura de datos que almacena pares de clave y valor y utiliza una clave hash para determinar la ubicación de almacenamiento de cada par. Esta estructura permite indexar y localizar elementos de manera eficiente a través de una función hash.

Funcionamiento

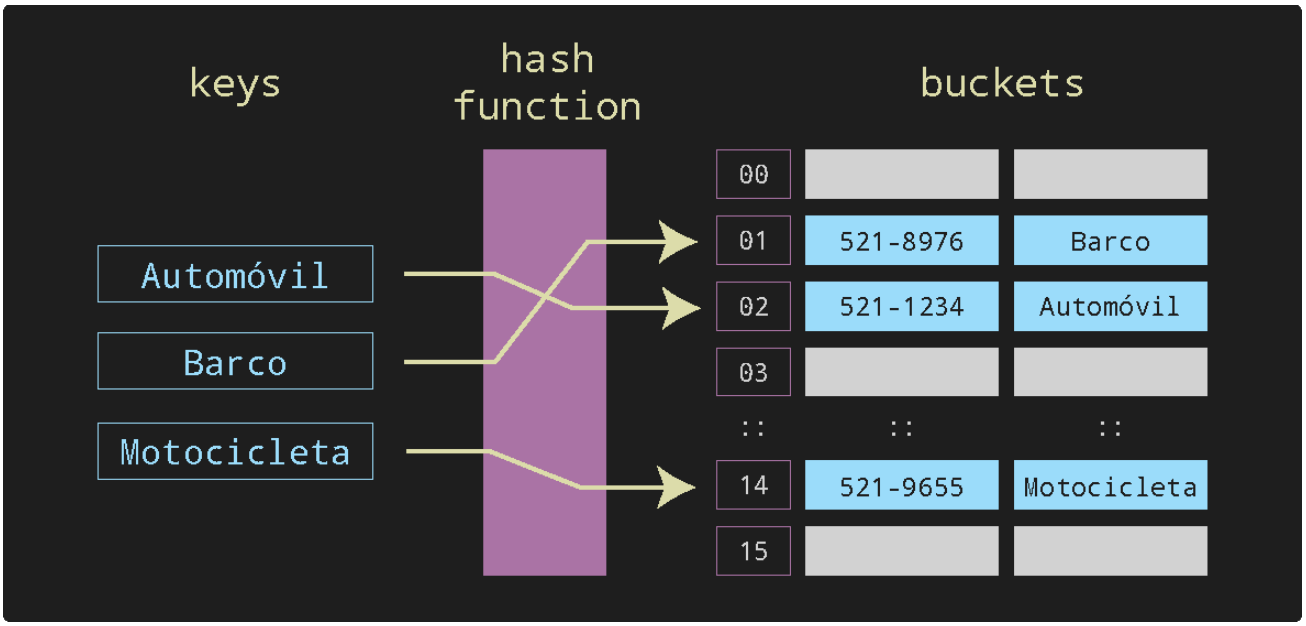
Almacenamiento de Pares (Clave, Valor):

- Cada elemento en un hashtable se almacena como un par de clave y valor.
- La clave se utiliza para calcular una posición única en la tabla a través de una función hash.

Función Hash:

- La función hash toma una clave y devuelve la posición en la tabla donde se almacenará el valor asociado.
- Ejemplo de una función hash simple:

```
def function_hash(clave):
  return hash(clave) % tamaño_tabla
```



*Un **bucket** contiene un par **clave-valor***

1. **Proceso de Inserción:**
   - Cuando se agrega un elemento, éste se coloca en un sector de almacenamiento en función del código hash de la clave.
   - Este código hash determina la ubicación exacta en la tabla donde se almacenará el par clave-valor.
2. **Búsqueda de Elementos:**
   - Las búsquedas utilizan su código hash para buscar en un sector de almacenamiento determinado solamente.
   - Esto reduce considerablemente la cantidad de comparaciones de clave necesarias para encontrar un elemento particular.
   - La eficiencia de búsqueda en hashtables es una de sus principales ventajas.

3. **Acceso a Elementos:**
    - Los elementos se pueden acceder a través de su clave utilizando la función hash para localizar su posición en la tabla.
    - En lenguajes de programación como C# y Java, los elementos de un hashtable se pueden manejar como objetos `DictionaryEntry`.

Ventajas

- **Eficiencia:** El acceso, inserción y eliminación de elementos son operaciones rápidas.
- **Reducción de Comparaciones:** Al utilizar el código hash, se minimiza la cantidad de comparaciones necesarias para encontrar un elemento, lo que mejora significativamente el rendimiento en búsquedas.
- **Flexibilidad:** Puede manejar grandes cantidades de datos con facilidad.
- **Útil para Grandes Cantidades de Información:** Los hashtables son ideales para manejar grandes volúmenes de datos debido a su eficiencia en operaciones básicas.

Desventajas

- **Colisiones:** Ocurren cuando dos claves diferentes producen el mismo valor hash. Esto se maneja con técnicas como la encadenamiento (listas enlazadas) o direccionamiento abierto.
- **Consumo de memoria:** Puede ser más costoso en términos de memoria comparado con otras estructuras de datos.
- **Acceso Ordenado Lento:** Debido a que los elementos se almacenan en posiciones pseudoaleatorias, el acceso ordenado a su contenido es bastante lento en comparación con otras estructuras de datos como árboles o listas enlazadas.

Aplicaciones

- Implementación de diccionarios y mapas.
- Tablas de búsqueda rápidas.
- Indexación de bases de datos.
- Caching de datos.

Ejemplo de Hashtable en C#

```
using System;
using System.Collections;
using System.Windows.Forms;

namespace EjemploHashtable
{
    public partial class Form1 : Form
    {
        private Hashtable tabla;

        public Form1()
        {
            InitializeComponent();
            tabla = new Hashtable();
        }

        private void buttonAgregar_Click(object sender, EventArgs e)
        {
            string clave = textBoxClave.Text;
            string valor = textBoxValor.Text;

            if (!tabla.ContainsKey(clave))
            {
                tabla.Add(clave, valor);
                listBoxElementos.Items.Add($"Clave: {clave}, Valor: {valor}");
                MessageBox.Show("Elemento agregado correctamente.");
            }
            else
            {
                MessageBox.Show("La clave ya existe.");
            }
        }

        private void buttonBuscar_Click(object sender, EventArgs e)
        {
            string clave = textBoxClave.Text;

            if (tabla.ContainsKey(clave))
            {
                string valor = (string)tabla[clave];
                MessageBox.Show($"Valor encontrado: {valor}");
            }
            else
            {
                MessageBox.Show("Clave no encontrada.");
            }
        }
    }
}
```

Métodos

Agregar Elementos (Add)

- **Método Add:**
  - Agrega un elemento con la clave y el valor especificados a la `Hashtable`.
  - La clave no puede ser nula, pero el valor sí.
  - Ejemplo:

```
Hashtable myHT = new Hashtable();
myHT.Add("uno", "Una");
myHT.Add("dos", "palabra");
```

- **Propiedad Item:**
  - También se puede usar la propiedad `Item` para agregar nuevos elementos, estableciendo el valor de una clave que no existe en la `Hashtable`.
  - Ejemplo:

```
Hashtable myHT = new Hashtable();
myHT["claveNoExistente"] = "nuevoValor";
```

  - Si la clave especificada ya existe en la colección, establecer la propiedad `Item` sobrescribirá el valor anterior.
  - Ejemplo:

```
myHT["dos"] = "nuevoValor";
```

Determinar la Existencia de Claves y Valores

- **Contains y ContainsKey:**
  - Determina si la `Hashtable` contiene una clave específica, devolviendo `true` si la clave existe o `false` si no.
  - Ejemplo:

```
Hashtable myHT = new Hashtable();
myHT.Add("dos", "palabra");
myHT.Add("tres", "más");
myHT.Add("cuatro", "otra");
Console.WriteLine(myHT.ContainsKey("dos")); // Salida: true
Console.WriteLine(myHT.ContainsKey("cinco")); // Salida: false
```

- **ContainsValue:**
  - Determina si la `Hashtable` contiene un valor específico. Devuelve `true` si el valor existe.
  - Ejemplo:

```
Hashtable myHT = new Hashtable();
myHT.Add("dos", "palabra");
myHT.Add("tres", "más");
myHT.Add("cuatro", "otra");
Console.WriteLine(myHT.ContainsValue("Palabra")); // Salida: false
Console.WriteLine(myHT.ContainsValue("palabra")); // Salida: true
```

Eliminar Elementos

- **Clear:**
  - Quita todos los elementos de la `Hashtable`.
  - Ejemplo:

```
Hashtable myHT = new Hashtable();
myHT.Add("dos", "palabra");
myHT.Add("tres", "más");
myHT.Add("cuatro", "otra");
myHT.Clear();
// Ahora myHT está vacío
```

- **Remove:**
  - Quita el elemento con la clave especificada de la `Hashtable`. Si no contiene un elemento con esa clave, la `Hashtable` no sufre ningún cambio.
  - Ejemplo:

```
Hashtable myHT = new Hashtable();
myHT.Add("dos", "palabra");
myHT.Add("tres", "más");
myHT.Add("cuatro", "otra");

myHT.Remove("Cuatro"); /* No ocurren cambios porque la clave
es sensible a mayúsculas y minúsculas */

myHT.Remove("cuatro"); // Quita el elemento cuya clave es "cuatro"
```